

ESpect First-Time Setup and Capture Guide

Version: 1.0

Date: 2026-09-18

Use this guide once for a new workstation or new VM copies. It installs the VM services, verifies the two endpoints, and explains the first real capture. After this guide succeeds, use [Docs/ESpect_Daily_Startup_Guide.md](#) for normal runs.

This guide has two distinct phases:

1. **One-time setup:** install packages, create `espect-agent.service`, and enable strongSwan.
2. **First capture:** start the Windows services, click the dashboard's main **Start capture** button, and verify the real PCAP and analysis.

Do not mix the diagnostic fallback commands into normal operation. They exist only for troubleshooting when the dashboard or backend is unavailable.

1. System layout

Component	Where it runs	Address or URL	Purpose
Windows host	Windows PowerShell	<code>C:\Users\Subhadeep\ESpect</code>	Frontend, backend, and analysis
VM1	Linux terminal	<code>192.168.160.128</code>	Controller, traffic coordinator, and capture endpoint
VM2	Linux terminal	<code>192.168.160.129</code>	Second IPsec endpoint and traffic peer

Both VM agents listen on TCP port `9000`. VM1's controller connects to both agents. The browser is not the packet source: VM1's `packet_capture.py` starts the real `tcpdump` capture and generates traffic through the two VMs.

Expected paths:

Windows: `C:\Users\Subhadeep\ESpect`
VMs: `/home/kali/ESpect`

2. Required software

Windows host

- Python and `ipsec_analyzer\backend\.venv`
- Node.js and npm
- The repository at `C:\Users\Subhadeep\ESpect`
- VMware or another hypervisor with both Linux VMs

Both Linux VMs

- System Python 3 with PyYAML and Scapy
- `tcpdump`

- strongSwan with `swanctl`
- sudo access without a password prompt while capture runs
- The repository copied to `/home/kali/ESpect`
- Experiment interface `eth1`

The current capture code uses `eth1` as `CAPTURE_INTERFACE`. Check it before the first run:

```
ip -br addr
ip -br link
```

If the experiment network is not `eth1`, update the interface in `testbed/sih-ipsec-analyzer_vm1/scripts/packet_capture.py` before running.

3. One-time VM installation

Run this once on each VM. It installs only missing packages, never runs a system upgrade, never removes installed packages, enables strongSwan to start automatically, and verifies the correct VM agent. Do not activate a VM virtual environment or run pip commands; the VM uses `/usr/bin/python3`.

First ensure the VM has temporary Internet access through a NAT adapter. The isolated experiment network on `eth1` cannot reach package servers. Then run the matching command:

On VM1:

```
cd /home/kali/ESpect
sudo bash testbed/setup_vm.sh vm1
```

On VM2:

```
cd /home/kali/ESpect
sudo bash testbed/setup_vm.sh vm2
```

The script uses `apt-get install --no-remove`, so it never removes packages. If package DNS is unavailable, it continues with the packages already installed and still creates the persistent agent service. Restore VM package-network access later and rerun the same command to add any missing package; it is safe to rerun.

After setup completes, the agent starts automatically. Do not create a VM virtual environment, run pip, or manually launch `run_agent.py` during normal operation.

4. Start strongSwan on both VMs

Run on VM1 and VM2:

```
sudo -v
sudo systemctl start strongswan
sudo systemctl is-active strongswan
which swanctl
sudo swanctl --list-conns
```

The service check must print `active`. If the service name differs:

```
systemctl list-units --type=service | grep -i strong
```

Do not continue until `swanctl` is installed and the service can be started.

5. Verify the VM network

On VM1:

```
ip -br addr
ping -c 1 192.168.160.129
```

On VM2:

```
ip -br addr
ping -c 1 192.168.160.128
```

Ping must succeed in both directions. Expected addresses are VM1 **192.168.160.128** and VM2 **192.168.160.129**.

6. Confirm the agents

The one-time installer creates and enables **espect-agent.service**, so the agent starts automatically whenever the VM boots. No terminal needs to remain open and you do not run **python3 scripts/run_agent.py** during normal startup.

On each VM:

```
sudo systemctl is-active espect-agent
sudo systemctl status espect-agent --no-pager -l
```

If it prints **inactive**, start it and check the result:

```
sudo systemctl enable espect-agent
sudo systemctl restart espect-agent
sudo systemctl is-active espect-agent
```

If it still does not print **active**, display the actual startup error:

```
sudo journalctl -u espect-agent --no-pager -n 80
```

If it prints **Unit espect-agent.service could not be found**, the one-time installer did not finish on this VM. Once the VM has package-network access, run the matching installer once again:

```
cd /home/kali/ESpect
sudo bash testbed/setup_vm.sh vm2
```

Use **vm1** on VM1. The installer is safe to rerun and does not remove packages.

Confirm both ports from VM1:

```
nc -zv -w 2 192.168.160.128 9000
nc -zv -w 2 192.168.160.129 9000
```

If **nc** is unavailable:

```
timeout 2 bash -c '</dev/tcp/192.168.160.128/9000' && echo VM1-agent-OK
timeout 2 bash -c '</dev/tcp/192.168.160.129/9000' && echo VM2-agent-OK
```

7. Start the Windows backend

Open PowerShell and check port **8000**:

```
Get-NetTCPConnection -LocalPort 8000 -State Listen -ErrorAction SilentlyContinue | Select-Object LocalAddress,LocalPort,Owni
```

If an old process is listening and it is safe to stop, use its displayed PID:

```
Stop-Process -Id <PID> -Force
```

Start the backend and keep this window open:

```
Set-Location 'C:\Users\Subhadeep\ESpect\ipsec_analyzer\backend'  
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Verify it from another PowerShell window. Use `curl.exe`, not plain `curl`:

```
curl.exe -sS --max-time 5 http://127.0.0.1:8000/health
```

Expected response:

```
{"status":"ok"}
```

8. Start the Windows frontend

Open another PowerShell window and keep it open:

```
Set-Location 'C:\Users\Subhadeep\ESpect\ipsec_analyzer\frontend'  
npm install  
npm run dev
```

Open:

```
http://127.0.0.1:5173
```

If Vite chooses another port, use the URL printed by Vite. The backend currently allows frontend ports `5173` and `5174`.

9. Confirm the capture configuration

On VM1:

```
cd /home/kali/ESpect/testbed/sih-ipsec-analyzer_vm1  
cat config/configuration.yaml
```

Expected default configuration:

```
ipsec_mode: transport  
encryption: aes128-gcm16  
integrity: none-aead  
dh_group: modp2048  
pfs: true  
ip_version: ipv4  
traffic_type: voip
```

Supported traffic types are `voip`, `whatsapp`, `email`, `web`, `icmp`, and `video`. Supported modes are `transport` and `tunnel`; supported IP versions are `ipv4` and `ipv6`.

To create a validated configuration with defaults:

```
cd /home/kali/ESpect/testbed/sih-ipsec-analyzer_vm1  
export PYTHONPATH=$PWD  
/usr/bin/python3 scripts/configurator.py --output config/configuration.yaml
```

10. Check sudo before capture

Run on both VMs:

```
sudo -v
sudo -n true && echo 'non-interactive sudo OK'
```

This matters because the capture helper starts `tcpdump` with `sudo -n`. If this fails, configure `sudo` for the lab user before continuing.

11. Run the real packet capture from the dashboard

After both `espect-agent` services are active, do not run the capture script manually during normal operation.

On Windows:

1. Start the backend and frontend from sections 7 and 8.
2. Open `http://127.0.0.1:5173`.
3. Open **Live capture**.
4. Click the main **Start capture** button once.
5. Wait for the capture workflow to complete.

The dashboard starts `packet_capture.py` on VM1 through the persistent agent, polls the real process, downloads the validated PCAP, and analyzes it automatically. The individual workflow rows are status indicators, not buttons.

Diagnostic fallback only

Use the following VM1 command only when the dashboard or backend is unavailable:

```
cd /home/kali/ESpect/testbed/sih-ipsec-analyzer_vm1
export PYTHONPATH=$PWD
sudo -v
/usr/bin/python3 scripts/packet_capture.py config/configuration.yaml 30 --output captures/live_capture.pcap
```

The arguments mean:

Argument	Meaning
<code>config/configuration.yaml</code>	IPsec experiment configuration
<code>30</code>	Capture and traffic duration in seconds
<code>--output captures/live_capture.pcap</code>	PCAP path on VM1

The script performs six stages:

1. Configure both agents.
2. Apply strongSwan IPsec configuration.
3. Start `tcpdump` on `eth1` before IKE negotiation.
4. Initiate IPsec from VM1.
5. Verify both IKE SAs and generate traffic.
6. Stop capture, terminate IPsec, download, and validate the PCAP.

The expected output file is:

```
/home/kali/ESpect/testbed/sih-ipsec-analyzer_vm1/captures/live_capture.pcap
```

12. Validate the PCAP on VM1

```
cd /home/kali/ESpect/testbed/sih-ipsec-analyzer_vm1
ls -lh captures/live_capture.pcap
file captures/live_capture.pcap
tcpdump -nn -r captures/live_capture.pcap | head -20
```

The file must exist and `tcpdump` must print packet records. On failure inspect:

```
cat debug/logs/tcpdump.log
```

For the default IPv4 configuration, the filter is:

```
host 192.168.160.128 and host 192.168.160.129 and (udp port 500 or udp port 4500 or ip proto 50)
```

13. Automatic PCAP transfer and analysis

When the dashboard workflow completes, it transfers the PCAP from VM1 through the agent connection and sends it to the analyzer automatically. No SCP, browser upload, or manual file selection is required during normal operation.

The following is a diagnostic fallback only if automatic transfer is unavailable.

Expected Windows destination:

```
C:\Users\Subhadeep\ESpect\testbed\sih-ipsec-analyzer_vm1\captures\live_capture.pcap
```

Example from PowerShell when SSH is enabled:

```
scp kali@192.168.160.128:/home/kali/ESpect/testbed/sih-ipsec-analyzer_vm1/captures/live_capture.pcap 'C:\Users\Subhadeep\ESpect\testbed\sih-ipsec-analyzer_vm1\captures\live_capture.pcap'
```

Replace `kali` with the actual VM username if different. Verify the Windows file:

```
Test-Path 'C:\Users\Subhadeep\ESpect\testbed\sih-ipsec-analyzer_vm1\captures\live_capture.pcap'
Get-Item 'C:\Users\Subhadeep\ESpect\testbed\sih-ipsec-analyzer_vm1\captures\live_capture.pcap' | Select-Object FullName, Length
```

14. Analyze the PCAP manually only as a fallback

1. Confirm the backend health response is `{"status":"ok"}`.
2. Open `http://127.0.0.1:5173`.
3. Open the analysis/upload view.
4. Select the Windows copy of `live_capture.pcap`.
5. Click `Analyze`.
6. Wait for the result before starting another upload.

The backend accepts `.pcap` and `.pcapng` through `/api/analyze`.

PowerShell API test:

```
$pcap = 'C:\Users\Subhadeep\ESpect\testbed\sih-ipsec-analyzer_vm1\captures\live_capture.pcap'
curl.exe -sS --max-time 120 -F "file=@$pcap" http://127.0.0.1:8000/api/analyze
```

Use a longer timeout for larger captures. To generate the PDF security report:

```
$pcap = 'C:\Users\Subhadeep\ESpect\testbed\sih-ipsec-analyzer_vm1\captures\live_capture.pcap'
curl.exe -sS --max-time 120 -F "file=@$pcap" http://127.0.0.1:8000/api/report -o 'C:\Users\Subhadeep\ESpect\Docs\ESpect_Report.pdf'
```

15. Normal shutdown

The capture script exits after its duration. If it is stuck, press **Ctrl+C** in its terminal. After the PCAP is copied, press **Ctrl+C** in each agent terminal, then in the backend and frontend terminals.

If a later run reports a stale IPsec SA, run on both VMs:

```
sudo swanctl --terminate --ike learning-vpn || true
sudo swanctl --list-sas
```

Restart both agents before retrying.

16. Troubleshooting

Placeholder path error

`/path/to/testbed/...` is not a real path. Use:

```
cd /home/kali/ESpect/testbed/sih-ipsec-analyzer_vm2
```

Agent service is not active

Check the service log:

```
sudo systemctl status espect-agent --no-pager -l
sudo journalctl -u espect-agent --no-pager -n 80
sudo systemctl restart espect-agent
```

If the log says `ModuleNotFoundError`, the one-time installer did not complete. Do not run pip repeatedly; run `sudo bash testbed/setup_vm.sh vm1` or `vm2` once after fixing VM package-network access.

Cannot communicate with an agent

Check ping, TCP port `9000`, the agent terminal, and any VM firewall. Both agents must be running.

tcpdump exited immediately

Check the interface and privileges:

```
ip -br link
sudo -n true
sudo tcpdump -i eth1 -nn -c 1
```

The current code requires `eth1` and tcpdump privileges.

IPsec configuration or initiation failure

Run on both VMs:

```
sudo systemctl is-active strongswan
sudo swanctl --list-conns
sudo swanctl --load-conns
sudo swanctl --load-creds
```

Check that both VMs use the same configuration and expected outer IPs.

IKE SA not established

```
sudo swanctl --list-sas
sudo journalctl -u strongswan --no-pager -n 80
```

Also verify UDP ports 500 and 4500 are allowed between the VMs.

Vite reports Unused args: 5173

Use:

```
npm run dev
```

Backend port 8000 is busy

```
Get-NetTCPConnection -LocalPort 8000 -State Listen | Select-Object OwningProcess  
Stop-Process -Id <PID> -Force
```

PowerShell asks for a Uri when using curl

Use `curl.exe` explicitly:

```
curl.exe -sS http://127.0.0.1:8000/health
```

Analysis times out

Check health first:

```
curl.exe -sS --max-time 5 http://127.0.0.1:8000/health
```

If health is blocked, restart Uvicorn. If health works, retry analysis with `--max-time 120` and watch the backend terminal.

17. Final success checklist

- [] VM1 is running at 192.168.160.128.
- [] VM2 is running at 192.168.160.129.
- [] Ping succeeds in both directions.
- [] strongSwan is active on both VMs.
- [] `expect-agent.service` is active on both VMs.
- [] VM1 and VM2 agents are listening on TCP 9000.
- [] VM1 can connect to both agent ports.
- [] `eth1` is the experiment interface.
- [] `sudo -n true` succeeds on both VMs.
- [] Backend health returns `{"status":"ok"}`.
- [] Frontend opens at `http://127.0.0.1:5173`.
- [] The capture script completes all six stages.
- [] `captures/live_capture.pcap` exists on VM1.
- [] `tcpdump -r` prints packets.
- [] The PCAP has been copied to Windows.
- [] The analyzer accepts the PCAP.
- [] The analysis or report has been saved.

18. Source files used

- `testbed/sih-ipsec-analyzer_vm1/scripts/packet_capture.py`
- `testbed/sih-ipsec-analyzer_vm1/scripts/run_agent.py`
- `testbed/sih-ipsec-analyzer_vm2/scripts/run_agent.py`
- `testbed/sih-ipsec-analyzer_vm1/src/controller/controller.py`
- `testbed/sih-ipsec-analyzer_vm1/src/agent/agent.py`
- `testbed/sih-ipsec-analyzer_vm2/src/agent/agent.py`
- `testbed/sih-ipsec-analyzer_vm1/config/configuration.yaml`
- `ipsec_analyzer/backend/app/main.py`
- `ipsec_analyzer/frontend/src/services/api.js`